

ARCHITECTURE

DocProcessor Architecture

Overview

Go module (`digital.vasic.docprocessor`) for loading project documentation, building structured feature maps, and tracking verification coverage. Part of the HelixQA ecosystem -- provides the "what to test" knowledge base that drives autonomous QA sessions.

Package Structure

```
pkg/  
  loader/      -- Document loading (Markdown, YAML) with recursive  
directory scanning  
  feature/     -- Feature map building with categories, screens,  
and test steps  
  coverage/    -- Thread-safe verification tracking with evidence  
and issue reporting  
  docgraph/    -- Document relationship graph with JSON/Mermaid  
export  
  llm/         -- LLM agent interface for AI-assisted feature  
extraction  
  config/      -- Configuration from .env files  
cmd/  
  docprocessor/ -- CLI entry point
```

Document Processing Pipeline

```
Project docs (Markdown, YAML)  
  |  
  Loader.LoadDir(path)  
  |  
  []Document (parsed sections, links, metadata)  
  |  
  FeatureMapBuilder.Build(documents)
```

```

    +-- LLMAgent.ExtractFeatures() (optional, AI-assisted)
    +-- Category classification
    +-- Screen association
    +-- Test step generation
    |
FeatureMap
    |
CoverageTracker.Track(featureMap)
    +-- Per-platform verification status
    +-- Evidence collection (screenshots, videos, logs)
    +-- Issue tracking
    |
CoverageReport

```

Key Types

Loader

Loader interface with `LoadDir()` and `LoadFile()`. Parses Markdown into Document structs containing Section objects (title, level, content, line number) and extracted links. Supports Markdown and YAML formats. Max file size: 10 MB.

Feature

Feature has a deterministic ID (`GenerateID(name)` -- slug with hash suffix for long names), category, associated screen, and test steps. Eight categories: format, ui, network, settings, storage, auth, editor, other.

`FeatureMapBuilder` constructs a `FeatureMap` from documents, optionally using an `LLMAgent` to extract features from unstructured text.

Coverage

`CoverageTracker` is thread-safe (`sync.RWMutex`). Tracks per-feature, per-platform verification state:

- **States:** unverified, verified, failed, skipped
- **Evidence:** screenshot path, video path + offset, log path, timestamp, description
- **Issues:** type (visual/ux/accessibility/functional/performance/crash), severity, evidence links

`CoverageReport` aggregates coverage percentages by platform and category.

DocGraph

Directed graph of document relationships. Nodes are documents, edges are references (links between docs). Thread-safe via `sync.RWMutex`. Exports to JSON and Mermaid diagram format.

LLM Agent

LLMAgent is an injected interface with no module-level dependency on LLMOrchestrator:

```
type LLMAgent interface {  
    ExtractFeatures(ctx context.Context, doc Document)  
        ([]RawFeature, error)  
}
```

Prompt templates in `pkg/llm/prompts.go` guide the LLM to extract structured feature data from documentation.

Key Design Decisions

- **Interface injection** for LLM: DocProcessor has zero dependency on LLMOrchestrator or any specific LLM SDK. The agent is provided at runtime.
- **Thread-safe tracking**: Both CoverageTracker and DocGraph use `sync.RWMutex` because HelixQA updates them concurrently from multiple device test sessions.
- **Deterministic IDs**: Feature IDs are derived from names via slugification + SHA256 hash suffix, ensuring stability across runs.
- **Max file size limit**: 10 MB prevents memory exhaustion on accidentally included binary files.